

Synthesis Lesson

Welcome. This is a demo lesson teaching digital circuits with RTL viewer using Yosys. Yosys is ideal for lessons because it is free and open source and thus allows inspection of all process steps under the hood to get from Verilog to a netlist which is uploaded onto our FPGA boards.

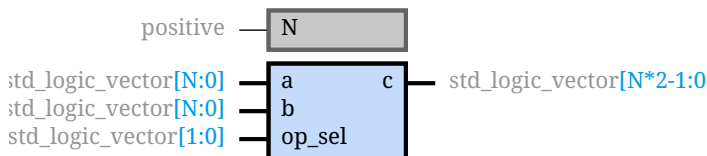
Introducing the ALU

We wish to build a simple "ALU" (arithmetic logic unit). In digital design we always start our design from a black box with inputs and outputs. This is the **interface** and designing it carefully is one of the key steps to making sure what we build is useful. Defining how the inputs and outputs interact over time with other components is done using a **protocol** (specification of how changes are handled) and is also important.

In this exercise the interface and protocol are both already specified. Your job will be writing the implementation of the interface, making sure that it *behaves* according to the the protocol we give you.

```
In [17]: %%bash
symbolator -i alu.vhd
```

```
/nix/store/8f0fnmbzy1zzjpnm78qzkfgpgm0fa2gk-symbolator-1.0.2/lib/python3.13/site-packages/symbolator/nucanvas/cairo_backend.py:17: PyGIWarning: Pango was imported without specifying a version first. Use gi.require_version('Pango', '1.0') before import to ensure that the right version gets loaded.
  from gi.repository import Pango as pango
/nix/store/8f0fnmbzy1zzjpnm78qzkfgpgm0fa2gk-symbolator-1.0.2/lib/python3.13/site-packages/symbolator/nucanvas/cairo_backend.py:18: PyGIWarning: PangoCairo was imported without specifying a version first. Use gi.require_version('PangoCairo', '1.0') before import to ensure that the right version gets loaded.
  from gi.repository import PangoCairo as pangocairo
('Scanning library:', '.')
Creating symbol for alu.vhd "alu"
-> alu-alu.svg
```



This is the interface for the ALU.

A functional description of the ALU might be:

The ALU takes in binary numbers `a` and `b`. It then performs an operation: it may:

0. Add `a` and `b`. (`a + b`)
1. Subtract `b` from `a`. (`a - b`)
2. Multiply `a` and `b`. (`a * b`)
3. Right shift `a` by `b`. (`a >> b`)

Which of these operations is performed is selected by using the `op_sel` input.

- If the binary number given to `op_sel` is the same as the decimal number 0, add.
- If the binary number given to `op_sel` is the same as the decimal number 1, subtract.

And so on.

Prior art

1. <https://pypi.org/project/sphinxcontrib-hdl-diagrams/>